

# UPSC Adaptive AI Orchestrator

## Implementation Roadmap — Architecture Review & Five-Phase Design

**Status:** Pre-Implementation Review Complete

**Purpose:** Authoritative reference for sequential module generation. Supersedes all prior phase definitions.

**Instruction:** Read this document fully before writing any module prompt. Every section is a prompt input constraint.

---

## Architecture Consistency Audit

Performed before phase design. All contradictions must be resolved before implementation begins.

---

### Contradiction Index

#### C-01 — score\_percentage Dual Semantics

**Component:** attempt\_history.score\_percentage

**Original assumption:** Single float column stores all evaluation results.

**Conflict:** MCQ uses 0.0–1.0 normalized scale. Mains uses 0–10 absolute scale. Both stored in the same column with no discriminator.

**Severity:** MEDIUM

**Type:** True Contradiction

**Resolution:** Normalize Mains score before storage. Store all scores on 0.0–1.0 scale. Mains raw score out of 10 is divided by 10.0 before writing. The raw score (0–10) is preserved inside detailed\_evaluation JSON for human display. Callers reading score\_percentage always get a comparable 0.0–1.0 value regardless of question type.

---

#### C-02 — expected\_seconds Undefined in P\_w Formula

**Component:** Certainty-Weighted Performance Vector, math\_utils.py

**Original assumption:** Formula defined as  $P_w = S \times (C_f \times (1 - \Delta T / T_{max}))$  where  $\Delta T = |response\_duration - expected\_seconds|$ .

**Conflict:** expected\_seconds is never defined, never stored, and has no source in any table or config file. The formula cannot be implemented.

**Severity:** BLOCKER

**Type:** True Contradiction — Missing Variable

**Resolution:** Add the following block to calibration\_config.yaml:

```
test_pacing:
  prelims_expected_seconds_per_question: 60      # 1800s / 30 questions
```

```
mains_expected_seconds_per_question: 450      # 1800s / 4 questions
csat_expected_seconds_per_question: 60
```

expected\_seconds is read from config at compute time based on question\_type. It is not stored per-question.

---

## C-03 — Memory Decay Formula Numerically Unstable

**Component:** Memory Decay Formula, math\_utils.py, topic\_progress

**Original formula:**  $I_{next} = I_{base} \times e^{(\alpha \times C_f \times (2 - D_t))}$  where  $D_t \in \{1..10\}$

**Conflict:** For  $D_t = 5$ : exponent = -3,  $I_{next} \approx 0.15$  days (3.6 hours). For  $D_t = 10$ :  $I_{next} \approx 0.001$  days (86 seconds). These values are not usable scheduling intervals. The formula is numerically unstable for any  $D_t > 3$  on the raw 1–10 scale.

**Severity:** BLOCKER

**Type:** True Contradiction — Formula Defect

**Resolution:** Two changes required:

1. Normalize  $D_t$  before applying formula. Let  $d_{norm} = (difficulty\_tier - 1) / 9$ , mapping  $D_t \in \{1..10\}$  to  $d_{norm} \in [0.0, 1.0]$ .
2. Incorporate `mistake_count` explicitly. The table has this field but the original formula does not use it. Define:  $mistake\_rate = mistake\_count / \max(times\_reviewed, 1)$   
 $effective\_d = \min(1.0, d_{norm} + 0.1 \times mistake\_rate)$
3. Revised formula:  $I_{next} = I_{base} \times e^{(\alpha \times C_f \times (0.5 - effective\_d))}$

This produces:

- $d_{norm} = 0.0$  (easiest),  $C_f = 1.0$ :  $I_{next} = I_{base} \times e^{0.5} \approx 4.95$  days
  - $d_{norm} = 0.5$  (medium),  $C_f = 1.0$ :  $I_{next} = I_{base} \times e^0 = 3.0$  days
  - $d_{norm} = 1.0$  (hardest),  $C_f = 1.0$ :  $I_{next} = I_{base} \times e^{(-0.5)} \approx 1.82$  days
  - $d_{norm} = 1.0$  (hardest),  $C_f = 0.5$ :  $I_{next} = I_{base} \times e^{(-0.25)} \approx 2.34$  days
1. Add hard bounds:  $I_{next} = \max(1.0, \min(30.0, I_{next}))$  to prevent sub-day or month-plus scheduling.
- 

## C-04 — P\_w Can Go Negative

**Component:** Certainty-Weighted Performance Vector

**Original formula:**  $P_w = S \times (C_f \times (1 - \Delta T / T_{max}))$

**Conflict:** When  $\Delta T > T_{max}$  (student takes longer than 120 seconds above expected pacing),  $(1 - \Delta T / T_{max}) < 0$ , resulting in  $P_w < 0$ . A negative  $P_w$  fed into the Elo formula produces a larger Elo penalty than a completely wrong answer ( $S=0$ ), which gives  $P_w=0$ . A slow correct

answer is penalized harder than an incorrect answer. This is educationally invalid.

**Severity:** HIGH

**Type:** True Contradiction — Formula Defect

**Resolution:** Clamp  $P_w$  after computation:  $P_w = \max(0.0, \min(1.0, P_w))$ . A correct answer never produces a negative performance score regardless of pacing.

---

## C-05 — Elo Calculation Misplaced in `generator.py`

**Component:** `generator.py::calculate_elo_update()`, `main.py::/submit-answer`

**Original assumption:** Elo is updated in `generator.py`.

**Conflict:** Elo updates happen at answer submission time, not question generation time. The `/submit-answer` endpoint in `main.py` calls `generator.calculate_elo_update()`. This means a module responsible for question generation also owns a function that has nothing to do with generation. This creates an implicit dependency between the orchestrator and the generator just to access a math function.

**Severity:** MEDIUM

**Type:** True Contradiction — Misplaced Responsibility

**Resolution:** Elo calculation is extracted into `math_utils.py`. `main.py` calls `math_utils.compute_elo_update()` directly. `generator.py` has no Elo function. See Section 7 for the full `math_utils.py` specification.

---

## C-06 — `evaluator.py` Interface Incompatible with `benchmark_runner.py`

**Component:** `evaluator.py`, `benchmark_runner.py`

**Original assumption:** `evaluator.py` accepts `question_id` as required input and retrieves the question blueprint from `question_bank`.

**Conflict:** `benchmark_runner.py` passes top-performing Mains essays through the evaluator. These essays have no associated `question_bank` entry and therefore no `question_id`. The evaluator as specified cannot accept benchmark inputs.

**Severity:** HIGH

**Type:** True Contradiction — Interface Incompatibility

**Resolution:** `evaluator.py::evaluate_mains_response()` accepts an optional `blueprint: Optional[Dict]` parameter. If `question_id` is provided, blueprint is retrieved from `question_bank`. If `blueprint` is provided directly, it is used without DB lookup. Exactly one of the two must be non-null. This preserves the existing standard evaluation path while enabling benchmark use.

---

## C-07 — `scraper.py` Lists `rag_store.py` as Dependency

**Component:** Module Dependency Graph, `scraper.py`

**Original assumption:** `Scraper` depends on `rag_store.py`.

**Conflict:** The scraper's ChromaDB-like dependency is deduplication — checking cosine similarity between incoming articles and existing records. But `rag_store.py` manages `syllabus_collection` and `pyq_collection`, neither of which stores news articles. Using these collections for article deduplication would be architecturally incorrect.

**Severity:** MEDIUM

**Type:** True Contradiction — Incorrect Dependency

**Resolution:** Scraper performs deduplication by computing embeddings on-the-fly using `sentence-transformers/all-MiniLM-L6-v2` directly (as a library import). It queries recent `current_affairs_feed` records from SQLite, embeds both the incoming and existing articles, computes cosine similarity, and merges or discards accordingly. `rag_store.py` is NOT a dependency of `scraper.py`. Scraper dependencies: `database.py`, `calibration.py`, `sentence-transformers` (local library), Cloud API.

---

## C-08 — `diagnostic.py` Unnecessarily Depends on `generator.py`

**Component:** Module Dependency Graph, `diagnostic.py`

**Original assumption:** `diagnostic.py` reads from `generator.py`.

**Conflict:** Diagnostic runs once, before the full system is operational. It does not need the 30-question composition pipeline, the Critic Agent, ChromaDB RAG, or backlog logic. Depending on `generator.py` chains the cold-start process to having Phases 1+2 fully complete. This increases the risk of the first run failing due to an unrelated infrastructure problem.

**Severity:** MEDIUM

**Type:** Acceptable Simplification → Redesign to Reduce Coupling

**Resolution:** `diagnostic.py` calls the Cloud API directly using `GeneratedQuestionSchema` for structured output. No dependency on `generator.py` or `rag_store.py`. Dependencies: `database.py`, `schemas.py`, `calibration.py`, Cloud API. This decouples cold-start from the generation infrastructure entirely.

---

## C-09 — `backlog_queue` Has No Content Source Type

**Component:** `backlog_queue` table, Composition Pipeline Step 4

**Original assumption:** Backlog rule applies 35% split "within each category" (static and CA separately).

**Conflict:** `backlog_queue.topic_type` stores only 'GS' or 'CSAT'. There is no field distinguishing whether a backlog GS topic should be served from static RAG or current affairs context. The category-level backlog split in Step 4 of the composition pipeline cannot be executed because the data model doesn't encode which content source the backlog item belongs to.

**Severity:** MEDIUM

**Type:** True Contradiction — Data Model Gap

**Resolution:** Add `source_type` field (Text, values: 'STATIC' or 'CA') to `backlog_queue`.

When a day is marked SKIPPED and its topic is written to the backlog, the original source type is preserved. The composition pipeline Step 4 reads this field to route backlog items to the correct content category.

---

## C-10 — `base_stability_index` Update Logic Undefined

**Component:** `topic_progress.base_stability_index`, Memory Decay Formula

**Original assumption:** `base_stability_index` is "updated after each review."

**Conflict:** No update formula, rule, or logic is defined anywhere. The column exists and is initialized to 3.0, but how it changes over time is completely unspecified. Without an update rule, the field is static and the memory decay formula always uses the same `I_base = 3.0`, making the formula partially inert.

**Severity:** HIGH

**Type:** True Contradiction — Missing Implementation Logic

**Resolution:** Define a simple exponential moving average update: After each attempt:

$$I\_base\_new = 0.9 \times I\_base + 0.1 \times I\_next$$
 This smoothly adjusts the topic's stability baseline toward the system's calculated optimal interval. Topics reviewed often at short intervals accumulate lower `I_base`; topics with long successful intervals accumulate higher `I_base`. The update is applied after `I_next` is computed, in the same write transaction.

---

## C-11 — `baseline_elo_rating` Update Rule Has No State Tracking

**Component:** `student_profile.baseline_elo_rating`, Recovery Velocity

**Original assumption:** `baseline_elo_rating` updates "after 20 consecutive attempts without Elo drop > 50 points."

**Conflict:** No field tracks "consecutive stable attempts." There is no counter, no state machine, and no reset trigger defined in any table. The update rule cannot be evaluated without this tracking state.

**Severity:** MEDIUM

**Type:** True Contradiction — Missing State Column

**Resolution:** Add `consecutive_stable_attempts` (Integer, default 0) to `student_profile`. After each attempt: if Elo dropped > 50 points, reset to 0. Otherwise, increment. When counter reaches 20, update `baseline_elo_rating` to `current_elo_rating` and reset counter to 0.

---

## C-12 — Recovery Velocity Division by Zero at Fast Recovery

**Component:** Recovery Velocity Formula  $V\_rec = \Delta Elo / \Delta t$

**Conflict:** If a student recovers their Elo within the same day ( $\Delta t = 0$ ), `V_rec` is undefined (division by zero).

**Severity:** LOW

**Type:** True Contradiction — Formula Edge Case

**Resolution:** Apply floor:  $\Delta t = \max(1, \text{days\_elapsed})$ . Recovery faster than one day is recorded as one day.

---

## C-13 — Elo Maximum Undefined

**Component:** Elo System

**Conflict:** A floor of 800 is defined. No ceiling is defined. With  $K=32$  and repeated correct answers, Elo can grow unboundedly. A student answering all difficulty-10 questions correctly would eventually reach Elo 5000+, making the system's difficulty targeting meaningless.

**Severity:** MEDIUM

**Type:** True Contradiction — Missing Bound

**Resolution:** Add `ceiling_rating: 2000` to `elo_system` config block. Elo is clamped to `[800, 2000]`. This matches the difficulty mapping range: difficulty 10 maps to `R_question = 2000`.

---

## Acceptable Simplifications (No Action Required)

- **APScheduler crash risk:** Accepted for V1. Archival jobs are maintenance, not correctness requirements.
  - **Diagnostic  $n=5$  reliability:** Accepted for V1. System self-corrects through adaptive Elo over subsequent sessions.
  - **Confidence gaming (always picking HIGH):** Unmitigated for V1. Anti-gaming mechanisms deferred to V2.
  - **No frontend specification:** API-only for V1. Confidence and duration are API request fields.
- 

## Deliberate Redesign Decisions (Intentional Departures)

- **`math_utils.py` extracted as new module:** Deliberate improvement. All math centralized and independently testable.
  - **`diagnostic.py` decoupled from `generator.py`:** Deliberate coupling reduction. Cold-start no longer blocked by generation infrastructure.
  - **`scraper.py` deduplication without `rag_store.py`:** Deliberate simplification. On-the-fly sentence-transformer embeddings are sufficient for V1 dedup at small scale.
- 

## Section 1: Dependency Analysis

### Corrected Module Dependency Graph

Phase 1 — No external dependencies

```
├── calibration_config.yaml
├── calibration.py           ← reads: config file only
├── models.py               ← reads: nothing
├── database.py             ← reads: models.py
├── schemas.py              ← reads: nothing
└── math_utils.py [NEW]     ← reads: calibration.py only
```

Phase 2 – Depends on Phase 1

- └─ rag\_store.py            ← reads: database.py, calibration.py
- └─ scraper.py             ← reads: database.py, calibration.py, sentence-transformers [library], Cloud API
- ← NOT: rag\_store.py [corrected]

Phase 3 – Depends on Phase 1 + Phase 2

- └─ generator.py           ← reads: database.py, schemas.py, calibration.py, math\_utils.py, rag\_store.py, Cloud API
- ← NOT: evaluate logic [boundary]
- └─ evaluator.py           ← reads: database.py, schemas.py, calibration.py, Cloud API
- ← NOT: generator.py, rag\_store.py [boundary]

Phase 4 – Depends on Phase 1 + 2 + 3

- └─ diagnostic.py          ← reads: database.py, schemas.py, calibration.py, Cloud API
- ← NOT: generator.py [corrected]
- └─ pdf\_exporter.py       ← reads: database.py, calibration.py
- ← NOT: generator.py, rag\_store.py [boundary]
- └─ safe\_mode.py          ← reads: database.py, calibration.py
- ← NOT: generator.py directly [routes to it via flag]

Phase 5 – Depends on all previous

- └─ main.py                ← reads: ALL modules above
- └─ benchmark\_runner.py   ← reads: database.py, evaluator.py, generator.py, calibration.py

## Hidden Dependencies Identified

| Hidden Dependency                     | Location                                              | Risk                                                                                                                                              |
|---------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| sentence-transformers model download  | First run of rag_store.py and scraper.py              | Requires network at initialization; must be cached locally before offline operation                                                               |
| Static PYQ pool in static_assets/pyq/ | safe_mode.py, benchmark_runner.py                     | These files must be manually provisioned before Phase 4 is operable. No code creates them.                                                        |
| ChromaDB collections populated        | generator.py (Critic Agent)                           | If pyq_collection is empty, Critic Agent semantic retrieval returns nothing. Generation continues but Critic quality drops. Should not hard-fail. |
| topper_copies/ files                  | benchmark_runner.py                                   | Must be manually provisioned. No code creates them.                                                                                               |
| Cloud API key                         | scraper.py, generator.py, evaluator.py, diagnostic.py | If not configured before Phase 2, all AI synthesis fails silently or hard-fails depending on error handling.                                      |

## Circular Dependencies

None detected in corrected graph.

## Modules Violating Orchestrator → Module Pattern

| Violation                          | Module              | Issue                                                              | Severity                                                                                                        |
|------------------------------------|---------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| generator.py → rag_store.py        | generator.py        | Generator calls RAG store directly without routing through main.py | ACCEPTED — pragmatic; routing through main.py would require main.py to pass large context objects as parameters |
| generator.py → math_utils.py       | generator.py        | Acceptable utility import                                          | ACCEPTED                                                                                                        |
| benchmark_runner.py → evaluator.py | benchmark_runner.py | Acceptable offline tool; not part of API server                    | ACCEPTED                                                                                                        |

## Section 2: Mathematical Validation

### 2.1 Elo Rating System

**Formula:**  $E = 1 / (1 + 10^{((R_q - R_{old}) / 400)})$  then  $R_{new} = R_{old} + K \times (P_w - E)$

**Validity:** Standard Elo formula. Mathematically correct for the intended purpose.

**Non-standard use:** Standard Elo uses  $S \in \{0, 0.5, 1\}$ . This system substitutes  $P_w \in [0, 1]$  continuously. This is a valid extension (commonly called "continuous Elo") but departs from classical assumptions. Document this explicitly in math\_utils.py.

#### Validated assumptions:

- $K = 32$  is appropriate for early-stage players with high volatility. Accept for V1.
- Floor at 800 prevents catastrophic failure states. Valid.
- Ceiling at 2000 (added via C-13) aligns with  $R_{question}$  maximum. Valid.

**Potential issue:**  $K = 32$  is constant. As a student accumulates hundreds of attempts, a constant  $K=32$  still allows large swings. Standard practice is to reduce  $K$  after a threshold number of games (e.g.,  $K=16$  after 100 attempts). Recommend: add `k_factor_decay_threshold: 100` and `k_factor_reduced: 16` to config for V2. Not required for V1.

**$R_{question}$  mapping:**  $R_{question} = difficulty\_tier \times 100 + 1000$ . At difficulty 1: 1100. At difficulty 10: 2000. This range [1100, 2000] against a student starting at 1200 is reasonable. The mapping should be explicitly hardcoded in math\_utils.py, not left as "recommended."



---

## 2.2 Certainty-Weighted Performance Vector

**Formula:**  $P_w = S \times (C_f \times (1 - \Delta T / T_{\max}))$  where  $\Delta T = |\text{response\_duration} - \text{expected\_duration}|$

**Pre-fix issues identified:**

- Expected\_seconds undefined → RESOLVED by C-02
- $P_w$  can go negative → RESOLVED by C-04

**Post-fix formula:**

```
P_w_raw = S × (C_f × (1 - |response_duration - expected_duration| / T_max))
P_w = max(0.0, min(1.0, P_w_raw))
```

**Validated behavior post-fix:**

- Correct + HIGH + on time:  $P_w = 1.0 \times (1.0 \times 1.0) = 1.0$  ✓
- Correct + LOW + on time:  $P_w = 1.0 \times (0.5 \times 1.0) = 0.5$  ✓
- Wrong answer (any confidence/pacing):  $P_w = 0 \times (...) = 0.0$  ✓
- Correct + HIGH + extreme overrun:  $P_w = \text{clamp}(\text{negative}) = 0.0$  ✓ (not penalized beyond wrong)

**Educational validity:** Penalizing guessing by confidence labeling is sound. However, there is no anti-gaming protection: a student who always selects HIGH confidence receives maximum  $P_w$  on all correct answers regardless of their actual certainty. This is a V1 accepted limitation.

**Asymmetry note:** A wrong answer with HIGH confidence gives  $P_w = 0$ . A wrong answer with LOW confidence also gives  $P_w = 0$ . The system does not reward epistemically honest wrong answers. Consider for V2:  $P_w$  for wrong+LOW = small positive (e.g., 0.1) to reward accurate self-assessment. Not required for V1.

---

## 2.3 Memory Decay Formula

**Pre-fix formula:**  $I_{\text{next}} = I_{\text{base}} \times e^{(\alpha \times C_f \times (2 - D_t))}$  where  $D_t \in \{1..10\}$

**Pre-fix issues:** Numerically unstable. See C-03.

**Post-fix formula:**

```
d_norm = (difficulty_tier - 1) / 9 # maps {1..10} to [0.0, 1.0]
mistake_rate = mistake_count / max(times_reviewed, 1)
effective_d = min(1.0, d_norm + 0.1 × mistake_rate)
I_next_raw = I_base × e^(alpha × C_f × (0.5 - effective_d))
I_next = max(1.0, min(30.0, I_next_raw)) # floor 1 day, ceiling 30 days
```

**Validated behavior:**

- Easy ( $d=0$ ) + high conf + no mistakes:  $I_{\text{next}} = 3 \times e^{0.5} \approx 4.95$  days ✓
- Medium ( $d=0.5$ ) + high conf:  $I_{\text{next}} = 3 \times e^0 = 3.0$  days ✓
- Hard ( $d=1.0$ ) + high conf + many mistakes: effective  $d$  approaches 1.0,  $I_{\text{next}} \approx 1.82$  days ✓
- Hard + low conf + many mistakes:  $I_{\text{next}}$  approaches 1.0 day (floor) ✓

#### **$I_{\text{base}}$ update rule (from C-10):**

$$I_{\text{base\_new}} = 0.9 \times I_{\text{base}} + 0.1 \times I_{\text{next}}$$

This is computed after  $I_{\text{next}}$  is determined and written in the same transaction as `topic_progress` update.

---

## **2.4 Recovery Velocity Index**

**Formula:**  $V_{\text{rec}} = \Delta E_{\text{lo}} / \Delta t$

**Post-fix (C-12):**  $\Delta t = \max(1, \text{days\_elapsed})$  to prevent division by zero.

**Validated:** Simple ratio. Mathematically trivial. The metric's value is in trending (is recovery getting faster over time?), not in absolute interpretation.

**Issue:**  $V_{\text{rec}}$  has no defined scale or target. A  $V_{\text{rec}}$  of 10 (Elo points per day) means nothing without context. Recommend storing all  $V_{\text{rec}}$  values historically in `experiment_runs` so trends are visible. For V1, the raw float is sufficient.

---

## **2.5 Critic Agent Scoring**

**Logic:** Per-dimension floor gates  $\rightarrow$  combined average gate. Both must pass.

**Gates:**

- `fact_check_verification`  $\geq 0.85$
- `semantic_authenticity`  $\geq 0.75$
- `distractor_plausibility`  $\geq 0.75$
- `blueprint_alignment`  $\geq 0.70$
- `combined_score` = `average(all four)`  $\geq 0.85$

**Validated:** The dual-gate logic correctly prevents a single very-low dimension score from being masked by high scores on others. The example (0.99/0.99/0.99/0.40 with combined = 0.84) fails Gate 2 correctly.

**Note on combined score:** With `fact_check`  $\geq 0.85$  as a hard floor and `combined`  $\geq 0.85$  as a second gate, a question passing `fact_check` at exactly 0.85 while all others score 0.85 would achieve combined = 0.85. This is technically a pass. The gates are calibrated correctly.

---

## Section 3: Educational Validation

### 3.1 Elo Adaptation Loop

**Validity:** Subject-level Elo adapted through spaced repetition is a well-established approach in computer-assisted learning.

**Risk — Early Volatility:** With  $K=32$  and starting Elo 1200, the first 10 questions can swing the rating by  $\pm 320$  points. This is large. The diagnostic partially mitigates this by providing a better starting point, but diagnostic accuracy is limited to  $n=5$  (see 3.3).

**Risk — Subject vs. Topic Granularity:** A student may be strong in "Constitutional Amendments" but weak in "Directive Principles" within Polity. Subject-level Elo conflates both. The `topic_progress` table captures topic-level data but has no Elo. This means the question difficulty targeting is at subject granularity, which is coarse. This is a known V1 limitation. Accept.

**Bias risk — No negative marking simulation:** UPSC Prelims has negative marking ( $-0.66$  per wrong answer on 2-mark questions). The Elo system penalizes wrong answers but not at the UPSC-specific ratio. The system measures relative proficiency, not UPSC marking fidelity. This is educationally appropriate — the goal is calibration, not mock scoring.

---

### 3.2 Mains Adversarial Evaluation

**Validity:** The dual-pass adversarial approach (blueprint check + adversarial cross-examination) correctly mimics how UPSC evaluators grade: first for structural completeness, then for analytical quality. Educationally sound.

**Risk — LLM Grading Bias:** LLMs have a documented tendency to reward well-structured, fluent text. The system instruction to penalize verbose prose and buzzwords mitigates this, but effectiveness depends entirely on prompt engineering quality. This cannot be validated architecturally — it must be validated empirically during benchmark testing.

**Risk — No Ground Truth Calibration at Submission Time:** When a student submits a Mains answer and receives a score, there is no mechanism to verify the LLM's score is calibrated correctly. Only `benchmark_runner.py` catches drift later. For V1, this is accepted. Recommend surfacing evaluator confidence level in the feedback to signal uncertainty.

**Risk — Blueprint Retrieval Dependency:** Pass 1 retrieves a `blueprint` from `question_bank.metadata.json`. If the question was served from the static PYQ pool (safe mode), the blueprint may not follow the structured `GeneratedQuestionSchema` format. Mains evaluation under safe mode may produce unreliable scores. Flag this in `safe_mode` handling.

---

### 3.3 Cold-Start Diagnostic Reliability

**Concern:** 5 questions per subject is statistically insufficient for reliable Elo initialization.

**Analysis:** With  $n=5$ , a student scoring 3/5 (60%) gets Elo 1200. A student scoring 4/5 (80%) also

gets Elo 1200 (both within 50%–85% band). A student guessing and getting 4/5 by luck gets 1200 rather than 1450. The bands are too coarse for the actual question count.

**Accepted for V1 because:** The diagnostic is explicitly a cold-start approximation. Adaptive Elo updates during subsequent daily sprints will self-correct within 5–10 sessions. This is documented behavior, not a bug.

**Recommendation:** In diagnostic.py terminal output, display the confidence interval alongside the assigned Elo: "Polity: 1200 Elo (60% accuracy,  $\pm 28\%$  margin at  $n=5$ )." This sets accurate expectations.

---

## 3.4 Syllabus Floor Guarantee

**Validity:** The 20% floor preventing hyper-personalization is educationally valid and well-motivated. Standard in adaptive testing to prevent coverage gaps.

**Interaction risk:** Floor questions contribute to the Elo system for their respective subjects, not necessarily the "target subject" of the day. A floor question drawn from Geography during a Polity session updates the Geography Elo. This is correct behavior but may be surprising. Document explicitly.

---

## 3.5 Backlog Scaling Risk

**Concern:** If multiple consecutive days are missed, backlog grows unboundedly. With 35% backlog allocation and a large backlog, the effective content per new topic shrinks.

**Example:** 5 skipped days  $\rightarrow 5 \times 2$  topics (GS + CSAT) = 10 backlog entries. The 35% allocation can only clear 10–11 questions per session. Backlog clearance is slow.

**Accepted for V1:** The `priority_weight` field in `backlog_queue` (increments daily) ensures oldest topics are cleared first. The 35% cap prevents backlog from completely dominating tests. Accept.

---

# Section 4: Five-Phase Roadmap

Each phase produces a working, independently testable subsystem. No phase requires the next phase to be functional.

---

## Phase 1 — Foundation

### Objective

Establish all persistent infrastructure, mathematical models, and data contracts. Zero external dependencies. Fully testable with unit tests and SQLite inspection only.

## Modules

- `calibration_config.yaml`
- `calibration.py`
- `models.py` + `database.py`
- `schemas.py`
- `math_utils.py` ← **New module, extracted from `generator.py`**

## Prerequisites

- Python environment with: SQLAlchemy, Pydantic v2, PyYAML
- Local file system write access to `UPSC_Agent_Data/`

## Inputs

- `calibration_config.yaml` file on disk
- No runtime inputs — this phase is infrastructure only

## Outputs

- `hub_database.db` initialized with all 9 tables + all schema additions from audit
- All Pydantic schemas importable and validating
- All math functions callable and returning correct values
- Config loading working with live-reload

## Interfaces Defined

See Section 5 — Phase 1.

## Key Implementation Notes

**`calibration_config.yaml` additions required by audit:**

```
test_pacing:
  prelims_expected_seconds_per_question: 60
  mains_expected_seconds_per_question: 450
  csat_expected_seconds_per_question: 60

elo_system:
  k_factor: 32
  base_rating: 1200
  floor_rating: 800
  ceiling_rating: 2000          ← NEW from C-13
```

**`models.py` additions required by audit:**

- `student_profile`: add `baseline_elo_rating` (Integer),  
    `recovery_velocity_score` (Float, default 0.0),  
    `consecutive_stable_attempts` (Integer, default 0)

- `topic_progress`: new table as specified (no Elo column)
- `backlog_queue`: add `source_type` (Text: 'STATIC' or 'CA')
- `attempt_history`: add `session_id` (Text), `confidence_level` (Text), `response_duration_seconds` (Float)
- `question_bank`: add `provenance_tags` (Text JSON), `generation_time_ms` (Integer), `tokens_consumed` (Integer), `critic_retry_count` (Integer, default 0)

#### **`math_utils.py` — all pure functions, no side effects:**

- `compute_difficulty_to_elo(difficulty_tier: int) → int` — hardcoded mapping
- `compute_expected_elo(R_old: int, R_question: int) → float`
- `compute_elo_update(R_old: int, K: int, P_w: float, E: float, floor: int, ceiling: int) → int`
- `compute_certainty_weighted_performance(S: int, confidence_level: str, response_duration: float, expected_duration: float, T_max: float) → float`
- `compute_memory_decay_interval(I_base: float, alpha: float, C_f: float, difficulty_tier: int, mistake_count: int, times_reviewed: int) → float`
- `update_stability_index(I_base: float, I_next: float) → float`
- `compute_recovery_velocity(delta_elo: int, delta_days: int) → float`

### **Risks**

- Schema conflicts if `models.py` is rebuilt before carefully applying all audit additions
- `math_utils.py` formula bugs undetected if unit tests are not written before Phase 2 begins

### **Dependencies**

None.

### **Success Criteria**

1. `init_db()` runs without error and all 9 tables are present
2. All Pydantic schemas reject invalid payloads with clear errors
3. `math_utils.py` unit tests pass for all edge cases (`P_w` clamping, Elo floor/ceiling, decay bounds)
4. `calibration.py` live-reload returns updated values without server restart
5. All custom exception classes are importable

## Estimated Complexity: LOW–MEDIUM

---

### Phase 2 — Data Ingestion

#### Objective

Build the two data collection systems that feed the intelligence layer. After this phase, ChromaDB is populated and news articles are being ingested automatically.

#### Modules

- `rag_store.py`
- `scraper.py` (two-mode)

#### Prerequisites

- Phase 1 complete
- `sentence-transformers/all-MiniLM-L6-v2` model downloaded and cached locally
- UPSC syllabus text files present in a local directory for ingestion
- PYQ JSON files (2013–2025) present in `static_assets/pyq/` for ingestion
- Cloud API key configured for AI synthesis step in scraper

#### Inputs

- One-time: `ingest_syllabus_documents(directory_path)` — called manually before first runtime
- One-time: `ingest_pyq_documents(directory_path)` — called manually before first runtime
- Runtime: `daily_news_scraper()` — called by APScheduler at 06:00
- Event-triggered: `document_ingestor(url, source_type)` — called on Tier 3 publication events

#### Outputs

- `vector_store/syllabus_collection/` populated with UPSC syllabus text chunks
- `vector_store/pyq_collection/` populated with PYQ embeddings
- `current_affairs_feed` receiving records daily

#### Interfaces Defined

See Section 5 — Phase 2.

## Key Implementation Notes

### `rag_store.py`:

- Embedding model `all-MiniLM-L6-v2` must be loaded once and reused (not reloaded per call)
- `retrieve_syllabus_chunks` and `retrieve_similar_pyqs` are the only runtime functions. The `ingest_*` functions are one-time setup only.
- If a ChromaDB collection does not exist at initialization, create it silently and return an empty result, not an error. The Critic Agent in `generator.py` should degrade gracefully if `pyq_collection` is empty.

### `scraper.py`:

- Two separate functions: `daily_news_scraper()` and `document_ingestor()`. They must not share execution logic.
- Deduplication uses `sentence-transformers` directly (not `rag_store.py`). Steps:
  1. Embed incoming article
  2. Query `current_affairs_feed` WHERE `fetches_at`  $\geq$  `now - 48h` from SQLite
  3. Embed each result
  4. Compute cosine similarity
  5. Merge if similarity  $\geq 0.88$ ; write new if below threshold
- Staging cache for Tier 2 consensus filter: this is an in-memory dict during the scraper run, not a database table. Articles failing consensus are simply not written in the current run; the next run's similarity check will naturally detect the same article if it appears again.
- Cloud API failure in the synthesis step: write the article to `current_affairs_feed` with `ai_synthesis = ''` and `syllabus_mapping = ''` rather than discarding. A null synthesis is recoverable; a lost article is not.
- `document_ingestor` is not automatically called — it requires an explicit trigger from the operator (API call or manual script). Do not register it on `APScheduler`.

## Risks

- `sentence-transformers` model requires ~90MB download. Must be cached before offline operation.
- Scraper sources (PIB, The Hindu, Indian Express) may change their URL patterns or rate-limit scrapers. These are maintenance risks, not implementation risks.
- Empty `pyq_collection` causes Critic Agent to operate without PYQ comparison context. Generation still works but quality scoring loses one dimension.



## Dependencies

- Phase 1 fully complete
- `sentence-transformers` library installed
- Cloud API key in environment

## Success Criteria

1. `initialize_collections()` creates both ChromaDB collections without error
2. After ingestion, `retrieve_syllabus_chunks("Fundamental Rights", n_results=5)` returns 5 non-empty strings
3. After ingestion, `retrieve_similar_pyqs("Which of the following...", n_results=3)` returns 3 records
4. `daily_news_scraper()` completes without error and writes at least one record to `current_affairs_feed`
5. Duplicate article (cosine similarity > 0.88) triggers merge, not a second write
6. Tier 2 article without consensus is NOT written to `current_affairs_feed`

## Estimated Complexity: MEDIUM-HIGH

---

## Phase 3 — Intelligence Core

### Objective

Build the question generation and Mains evaluation engines. This is the highest-complexity phase. After this phase, the system can produce 30-question papers and grade subjective answers without the API server.

### Modules

- `generator.py`
- `evaluator.py`

### Prerequisites

- Phase 1 + Phase 2 complete
- Cloud API key configured and active
- `pyq_collection` populated in ChromaDB (for Critic Agent)
- `current_affairs_feed` has at least some records (for CA question generation)

### Inputs

`generator.py`:

- `subject_id` (str), `test_type` (str), `session_id` (UUID str)
- Reads from: `student_profile`, `topic_progress`, `backlog_queue`, `daily_study_log`, `current_affairs_feed`
- Calls: `rag_store.retrieve_syllabus_chunks()`, `rag_store.retrieve_similar_pyqs()`, Cloud API (3× async)

#### **evaluator.py:**

- `question_id` (Optional[str]), `student_response` (str), `confidence_level` (str), `response_duration_seconds` (float), `blueprint` (Optional[Dict])
- Reads from: `question_bank` (if `question_id` provided)
- Calls: Cloud API (2× sequential — Pass 1 then Pass 2)

## **Outputs**

#### **generator.py:**

- Writes 30 question records to `question_bank` with full metadata and `provenance_tags`
- Returns list of `question_id` strings

#### **evaluator.py:**

- Writes evaluation record to `attempt_history`
- Returns score (float, normalized 0.0–1.0) and `detailed_evaluation` (markdown str)

## **Interfaces Defined**

See Section 5 — Phase 3.

## **Key Implementation Notes**

### **generator.py — Composition Pipeline (must be annotated in code):**

```
STEP 1: FLOOR GUARANTEE (execute first, always)
    floor_count = round(30 × 0.20) = 6
    floor_topics = random_draw_from_full_syllabus(floor_count)
    RULE: overlap with today's topic is ALLOWED. Do not redraw.
    RULE: content type (CA or static) follows the topic's natural category.

STEP 2: RECALCULATE REMAINING QUOTA DYNAMICALLY
    remaining = 30 - floor_count = 24

STEP 3: CONTENT TYPE SPLIT on remaining quota only
    static_count = round(24 × 0.60) = 14 (approximately)
    ca_count = 24 - static_count = 10 (approximately)

STEP 4: BACKLOG RULE within each content category
    IF backlog_queue non-empty:
        static_backlog = round(static_count × 0.35)
        static_today = static_count - static_backlog
        ca_backlog = round(ca_count × 0.35)
        ca_today = ca_count - ca_backlog
        (use backlog_queue.source_type to route each entry to correct category)
```

```
ELSE:
    static_today = static_count, ca_today = ca_count, all backlog counts = 0
ASSERT: floor_count + static_today + static_backlog + ca_today + ca_backlog ==
30
```

### **generator.py — Critic Agent pipeline:**

1. Generate draft via Cloud API
2. `rag_store.retrieve_similar_pyqs(draft.question_text, n_results=3)` → similar PYQs
3. Cloud API call: evaluate draft against PYQs → returns `CriticEvaluationSchema`
4. Check Gate 1 (per-dimension floors) AND Gate 2 (combined  $\geq 0.85$ )
5. If either gate fails → regenerate (max 3 retries)
6. If 3 retries exhausted → pull from `static_assets/pyq/` flat JSON pool
7. Store `critic_agent_consensus_score` in `provenance_tags`

**generator.py — Elo update:** `math_utils.compute_elo_update()` is called from here but only as a library import — the function is defined in `math_utils.py`, not here. This is a utility call, not a generator responsibility.

### **evaluator.py — Dual-pass:**

- Pass 1 and Pass 2 are sequential, not parallel (Pass 2 input depends on Pass 1 output)
- If `question_id` is `None` and `blueprint` is also `None` → raise `EvaluationFailure`
- Score stored as `score_percentage = raw_score / 10.0` (normalized to  $[0, 1]$  per C-01 resolution)
- Raw score (0–10) preserved in `detailed_evaluation` JSON string for display purposes

**generator.py — System Intent Header:** Every test generates a human-readable composition summary string: "Today's Test Architecture: 6 Floor (Random Syllabus), 9 Static Core [Polity: Adaptive Target], 5 Static Backlog [Economy: Recovery], 6 CA Today [Polity: Current], 4 CA Backlog [Economy: Recovery]" This string is returned alongside the question list and written to the first question's metadata or a separate `test_session` record.

## **Risks**

- **Highest-risk phase in the system.** Async generation with Critic Agent loop has multiple failure modes.
- Critic Agent retry loop: 3 retries  $\times$  3 workers  $\times$  10 questions = potentially 90 cloud API calls per test paper in worst case. Monitor `critic_retry_count` in `provenance_tags`.
- Cloud API structured output failures: even with Pydantic validation, LLMs occasionally

produce malformed JSON. The validation layer must reject and retry, not crash.

- Pass 1 → Pass 2 state passing in evaluator: Pass 2 must receive Pass 1 output in the context, not just the original response. Prompt construction must be explicit.
- Empty `current_affairs_feed`: if no records exist (system bootstrapping), CA questions cannot be generated. Fallback: treat CA slots as additional static slots.

## Dependencies

- Phase 1 + Phase 2 complete
- `pyq_collection` and `syllabus_collection` populated
- Cloud API active

## Success Criteria

1. `build_composition_plan()` returns allocation summing to 30 in all backlog/no-backlog scenarios
2. `generate_full_adaptive_exam()` produces exactly 30 questions, all passing schema validation
3. `critic_retry_count` in `provenance_tags` is populated correctly
4. Safe mode fallback activates when Cloud API call fails during generation (not a crash)
5. `evaluate_mains_response()` produces a score in [0.0, 1.0] and populated `detailed_evaluation`
6. `evaluate_mains_response()` with direct `blueprint` dict works (for benchmark compatibility)
7. All 30 questions written to `question_bank` before endpoint returns

## Estimated Complexity: HIGH

---

## Phase 4 — Delivery Layer

### Objective

Build all user-facing output tools: cold-start calibration, document export, and the operational circuit breaker. After this phase, the system can fully onboard a new user and produce all three PDF documents.

### Modules

- `diagnostic.py`
- `pdf_exporter.py`
- `safe_mode.py`

## Prerequisites

- Phase 1 + Phase 2 + Phase 3 complete
- `static_assets/pyq/` populated with PYQ JSON files (for `safe_mode` fallback)
- ReportLab library installed
- `exports/` directory writable

## Inputs

### **diagnostic.py:**

- No runtime inputs — one-time execution script
- Reads from: nothing (generates its own questions via Cloud API)

### **pdf\_exporter.py:**

- `session_id` (str) for question paper and answer key
- `date_str` (str, YYYY-MM-DD) for current affairs briefing

### **safe\_mode.py:**

- No inputs — reads state from Cloud API ping and `current_affairs_feed`

## Outputs

### **diagnostic.py:**

- Writes calibrated Elo to `student_profile` (5 rows, one per subject)
- Writes `baseline_elo_rating`, `weakness_tags` to `student_profile`
- Initializes `topic_progress` records for detected weak topics
- Prints summary to terminal

### **pdf\_exporter.py:**

- `Question_Paper.pdf` → `exports/`
- `Answer_Key_and_Analysis.pdf` → `exports/`
- `Daily_Current_Affairs_Briefing.pdf` → `exports/`

### **safe\_mode.py:**

- Returns `HealthStatus` enum
- Returns fallback question list when `SAFE_MODE_ACTIVE`
- Writes safe mode event to `experiment_runs`

## Interfaces Defined

See Section 5 — Phase 4.

## Key Implementation Notes

### **diagnostic.py:**

- Calls Cloud API directly for question generation using `GeneratedQuestionSchema`
- Does NOT use `generator.py`, `rag_store.py`, or the composition pipeline
- 5 questions per subject × 5 subjects = 25 questions total
- MCQ accuracy thresholds: >85% → 1450, 50–85% → 1200, <50% → 950
- Pacing analysis: record `epoch_start` when question displayed, `epoch_end` when answer submitted
- Weakness tags written as JSON array: `["Weak Core Memory", "Missing Structural Keywords"]`
- `baseline_elo_rating` = calibration Elo. This is the only unconditional write to this field.
- Terminal output must display per-subject results as they are written (not batch at end)

### **pdf\_exporter.py:**

- Watermark parameters: text "THE FELLOW ASPIRANT", font size 52, opacity `setFillAlpha(0.15)`, rotation 45°
- Watermark is applied to canvas before text is drawn on every page
- Watermark must be re-applied after every `showPage()` call
- Cursor management: `cursor_y < 120` → trigger `showPage()`, reset cursor, re-apply watermark
- Question Paper: question stems + MCQ options only. No answers, explanations, or trap analysis.
- Answer Key: per-question — bold answer header, "Core Concept Explained" section, "Trap Analysis & Elimination Strategy" block
- Briefing: bold article title → syllabus mapping sub-header → `ai_synthesis` body. Fetch `WHERE DATE(fetched_at) = date_str`.
- Mains blank space: 90 points below question stem in Question Paper

### **safe\_mode.py:**

- Health check: (1) ping Cloud API with minimal test call, (2) query `current_affairs_feed WHERE fetched_at >= now - 24h`
- `SAFE_MODE_ACTIVE = True` if: API timeout, API 5xx, or zero records in last 24h
- Fallback reads from `static_assets/pyq/prelims/` flat JSON files only — no ChromaDB
- Safe mode flag is in-memory (not persisted between requests). Each new request re-checks

health.

- Safe mode activation logged to `experiment_runs` with timestamp and reason
- Warning string returned: "Safe Mode Engaged: API limits reached. Serving offline static database."

## Risks

- `diagnostic.py` Cloud API failure during cold-start is a critical failure — the student cannot onboard. Must handle gracefully with retry or partial completion (write whatever was computed before failure).
- `pdf_exporter.py` cursor management is error-prone. ReportLab canvas state is not reset automatically across function calls. Test multi-page output explicitly.
- `safe_mode.py` static pool empty if not provisioned. This is an operational risk, not a code risk. Document the provisioning requirement.
- Mains `blueprint` from PYQ files (safe mode) may lack structured JSON format expected by evaluator Pass 1. Flag in evaluator: if blueprint is unstructured, skip Pass 1, run Pass 2 only and note reduced confidence in feedback.

## Dependencies

- Phase 1 + Phase 2 + Phase 3 complete
- Static PYQ pool manually provisioned in `static_assets/pyq/`
- Cloud API active (diagnostic only — `pdf_exporter` and `safe_mode` work without cloud)

## Success Criteria

1. `run_diagnostic()` writes 5 `student_profile` records with non-default Elo values
2. `generate_question_paper(session_id)` produces a valid PDF at the correct path
3. Watermark present on every page of every document
4. Page break correctly re-applies watermark
5. `health_check()` returns DEGRADED status within 5 seconds of Cloud API being unreachable
6. Fallback questions are served from `static_assets/pyq/` without any ChromaDB call
7. Safe mode activation is logged to `experiment_runs`

## Estimated Complexity: MEDIUM

---

## Phase 5 — Orchestration & Observability

### Objective

Wire all modules into a running FastAPI server with scheduled background jobs and drift detection. After this phase, the system is fully operational end-to-end.

### Modules

- `main.py`
- `benchmark_runner.py`

### Prerequisites

- All previous phases complete and tested
- All modules importable without error
- APScheduler library installed

### Inputs

#### **main.py:**

- HTTP requests on all defined routes

#### **benchmark\_runner.py:**

- No inputs — standalone script
- PYQ held-out set in `static_assets/pyq/`
- Topper copies in `static_assets/topper_copies/`

### Outputs

#### **main.py:**

- Running FastAPI server on configured port
- APScheduler running: `daily_news_scraper` at 06:00, `archival_hot_to_warm` at 02:00, `archival_warm_to_cold` Sunday 03:00

#### **benchmark\_runner.py:**

- Terminal output with alignment scores
- Drift warnings printed if thresholds exceeded
- All results written to `experiment_runs` table

### Interfaces Defined

See Section 5 — Phase 5.



## Key Implementation Notes

**main.py** — All business logic DELEGATED to spokes. **main.py** is routing only:

- POST /generate-test/prelims → `safe_mode.health_check()` → `generator.build_composition_plan()` → `generator.generate_full_adaptive_exam()` → return question list + session\_id + System Intent Header
- POST /submit-answer → fetch question from `question_bank` → `math_utils.compute_certainty_weighted_performance()` → `math_utils.compute_elo_update()` → update `student_profile` and `topic_progress` → compute pacing std dev → write `attempt_history`
- POST /evaluate/mains → `evaluator.evaluate_mains_response()` → write `attempt_history` → return score + feedback
- POST /generate-test/export → `pdf_exporter.generate_question_paper()` + `pdf_exporter.generate_answer_key()` → return file paths
- POST /generate-briefing/export → `pdf_exporter.generate_briefing()` → return file path

**main.py** — Startup event:

```
→ init_db()
→ load_config()
→ scheduler.add_job(daily_news_scraper, 'cron', hour=6)
→ scheduler.add_job(archival_hot_to_warm, 'cron', hour=2)
→ scheduler.add_job(archival_warm_to_cold, 'cron', day_of_week='sun', hour=3)
→ scheduler.start()
```

**main.py** — **Error handling:** Every endpoint wraps its spoke calls in try/except for each custom exception class. Returns structured JSON error payloads, never raw 500 responses. Error payload: `{"error_code": "GenerationFailure", "message": "...", "safe_mode_available": true}`.

**main.py** — **/submit-answer Psychological Drift Check:** After writing to `attempt_history`, query `attempt_history WHERE session_id = ?` and compute std dev of `response_duration_seconds`. If  $> \text{pacing\_std\_dev\_threshold}$  (0.40) → append `UserBehaviorFlag` to response payload. This is a warning flag, not a hard stop.

**benchmark\_runner.py:**

- Standalone script. Not imported by `main.py`. Not part of the API server.
- Topper copy evaluation uses `evaluator.evaluate_mains_response(question_id=None, blueprint=some_dict)` — requires the blueprint dict to be embedded in the topper copy data files or generated on-the-fly
- Drift threshold: topper copies scoring below 8.5/10 (0.85 normalized) → "System Drift"

Warning: Evaluation calibration degraded"

- All scores written to `experiment_runs` with `config_version` and `engineering_notes`

**APScheduler crash risk note:** Acknowledged. If the FastAPI process crashes, APScheduler dies with it. Archival jobs will not run until the server restarts. This is a maintenance risk, not a correctness risk. For V1, this is acceptable. The consequence is accumulating `detailed_evaluation` text beyond the 30-day window, which increases DB size but does not corrupt data.

## Risks

- Integration errors: `main.py` importing all modules simultaneously may reveal hidden import-time side effects. Each module must be importable without triggering file operations or network calls at import time.
- APScheduler job conflicts: if a previous scraper job is still running when the next one fires (e.g., Cloud API is slow), APScheduler may run two concurrent scrapers. Mitigate with `max_instances=1` on the scheduler job definition.
- Session ID collision: UUIDs are collision-resistant but not impossible. Accept for V1 single-user.

## Dependencies

- All previous phases complete

## Success Criteria

1. Server starts without error and all 5 routes respond
2. `POST /generate-test/prelims` returns 30 questions + `session_id` + System Intent Header within acceptable time
3. `POST /submit-answer` returns updated Elo and writes to `attempt_history`
4. APScheduler fires `daily_news_scraper` at 06:00 without manual trigger
5. `benchmark_runner.py` runs to completion and writes results to `experiment_runs`
6. Safe mode flag activates correctly and fallback questions are served under simulated API failure

**Estimated Complexity: MEDIUM-HIGH**

---

## Section 5: Interfaces Per Phase

### Phase 1 Interfaces

#### calibration.py

```
get_config() → CalibrationConfig      # cached singleton; reads YAML once
reload_config() → CalibrationConfig    # force fresh parse; invalidates cache
```

Failure: CalibrationFailure if YAML file missing or malformed.

#### database.py

```
get_session() → contextmanager[Session]  # yields SQLAlchemy session; auto-
commits or rolls back
init_db() → None                        # idempotent; creates all tables if
not present
```

Failure: DatabaseError (SQLAlchemy built-in) if disk full or permissions denied.

#### schemas.py

Five Pydantic v2 model classes. No functions. Instantiated by callers. Failure: ValidationError (Pydantic built-in) on invalid input.

#### math\_utils.py

```
compute_difficulty_to_elo(difficulty_tier: int) → int
    Input:  difficulty_tier ∈ {1..10}
    Output: R_question = difficulty_tier × 100 + 1000 ∈ {1100..2000}
    Failure: ValueError if out of range

compute_expected_elo(R_old: int, R_question: int) → float
    Input:  R_old ∈ {800..2000}, R_question ∈ {1100..2000}
    Output: E ∈ (0.0, 1.0)
    Failure: none (math is always valid)

compute_elo_update(R_old: int, K: int, P_w: float, E: float, floor: int,
ceiling: int) → int
    Input:  R_old, K=32, P_w ∈ [0.0, 1.0], E ∈ (0.0, 1.0), floor=800,
ceiling=2000
    Output: R_new ∈ {800..2000}
    Failure: none (clamped)

compute_certainty_weighted_performance(
    S: int,
    confidence_level: str,
    response_duration_seconds: float,
    expected_duration_seconds: float,
    T_max: float
) → float
    Input:  S ∈ {0, 1}, confidence_level ∈ {'HIGH', 'MEDIUM', 'LOW'},
           response_duration ≥ 0, expected_duration > 0, T_max = 120
    Output: P_w ∈ [0.0, 1.0] (clamped)
    Failure: ValueError if confidence_level not in valid set

compute_memory_decay_interval(
    I_base: float,
    alpha: float,
```

```

    C_f: float,
    difficulty_tier: int,
    mistake_count: int,
    times_reviewed: int
) → float
    Input:  I_base > 0, alpha = 1.0, C_f ∈ {0.5, 0.75, 1.0},
            difficulty_tier ∈ {1..10}, mistake_count ≥ 0, times_reviewed ≥ 0
    Output: I_next ∈ [1.0, 30.0] (bounded)
    Failure: none (bounded)

update_stability_index(I_base: float, I_next: float) → float
    Input:  I_base > 0, I_next > 0
    Output: I_base_new = 0.9 × I_base + 0.1 × I_next
    Failure: none

compute_recovery_velocity(delta_elo: int, delta_days: int) → float
    Input:  delta_elo ≥ 0, delta_days ≥ 0
    Output: V_rec = delta_elo / max(1, delta_days)
    Failure: none (floor on delta_days prevents division by zero)

```

---

## Phase 2 Interfaces

### rag\_store.py

```

initialize_collections() → None
    Side effect: creates ChromaDB collections if they do not exist
    Failure: RAGFailure if ChromaDB directory is not writable

ingest_syllabus_documents(directory_path: str) → int
    Input:  path to directory containing .txt syllabus files
    Output: count of chunks ingested
    Side effect: populates syllabus_collection
    Failure: RAGFailure if directory not found

ingest_pyq_documents(directory_path: str) → int
    Input:  path to static_assets/pyq/ containing year JSON files
    Output: count of PYQ records ingested
    Side effect: populates pyq_collection
    Failure: RAGFailure if directory not found

retrieve_syllabus_chunks(topic: str, n_results: int = 5) → List[str]
    Input:  topic string, n_results ≥ 1
    Output: list of relevant text chunks (may be empty if collection
unpopulated)
    Failure: RAGFailure only on ChromaDB internal error; empty list otherwise

retrieve_similar_pyqs(question_text: str, n_results: int = 3) → List[Dict]
    Input:  question text string, n_results ≥ 1
    Output: list of PYQ dicts from pyq_collection (may be empty)
    Failure: RAGFailure only on ChromaDB internal error; empty list otherwise

```

### scraper.py

```

daily_news_scraper() → None
    Side effect: writes approved articles to current_affairs_feed
    Failure: CurrentAffairsFailure logged but does not raise – scraper must not
crash the scheduler

document_ingestor(url: str, tier3_source_type: str) → None

```

Input: url to full document, source type label  
Side effect: writes sections to current\_affairs\_feed  
Failure: CurrentAffairsFailure logged but does not raise

---

## Phase 3 Interfaces

### generator.py

```
build_composition_plan(
    subject_id: str,
    test_type: str,
    total_questions: int = 30
) → CompositionPlan
Input: valid subject_id from student_profile, test_type ∈
{'PRELIMS_GS', 'CSAT'}
Output: CompositionPlan dataclass:
    {floor_count, static_today, static_backlog, ca_today, ca_backlog}
    assert sum == total_questions
Side effect: reads backlog_queue, daily_study_log from DB
Failure: GenerationFailure if subject_id not found

generate_full_adaptive_exam(
    plan: CompositionPlan,
    session_id: str
) → List[str]
Input: CompositionPlan, UUID session_id
Output: list of question_id strings (len == plan.total_questions)
Side effect: writes all questions to question_bank
Failure: GenerationFailure if cloud API unreachable and static pool
exhausted

run_critic_agent(
    draft_question: GeneratedQuestionSchema,
    similar_pyqs: List[Dict]
) → CriticEvaluationSchema
Input: draft question object, 0-3 similar PYQ records
Output: CriticEvaluationSchema with combined_score and optional
rejection_reason
Side effect: none (does not write to DB)
Failure: GenerationFailure if cloud API unreachable
```

### evaluator.py

```
evaluate_mains_response(
    student_response: str,
    confidence_level: str,
    response_duration_seconds: float,
    question_id: Optional[str] = None,
    blueprint: Optional[Dict] = None
) → EvaluationResult
Input: Exactly one of question_id or blueprint must be non-null
Output: EvaluationResult dataclass:
    {score_normalized: float, # 0.0-1.0
     raw_score: float, # 0.0-10.0
     detailed_evaluation: str, # markdown
     pass1_score: float,
     pass2_score: float}
Side effect: writes to attempt_history
Failure: EvaluationFailure if cloud API unreachable
```

ValueError if both question\_id and blueprint are None

---

## Phase 4 Interfaces

### diagnostic.py

```
run_diagnostic() → None
  Input:  none
  Output: none (all output written to DB and terminal)
  Side effect:
    - writes 5 student_profile records with calibrated Elo
    - writes baseline_elo_rating per subject
    - writes weakness_tags per subject
    - initializes topic_progress for weak topics
  Failure: CalibrationFailure if cloud API unreachable (partial writes
preserved)
```

### pdf\_exporter.py

```
generate_question_paper(session_id: str) → str
  Input:  session_id with questions in question_bank
  Output: absolute file path to Question_Paper.pdf
  Failure: FileNotFoundError if exports/ not writable; ValueError if no
questions found

generate_answer_key(session_id: str) → str
  Input:  session_id with questions in question_bank
  Output: absolute file path to Answer_Key_and_Analysis.pdf
  Failure: same as above

generate_briefing(date_str: str) → str
  Input:  date_str in YYYY-MM-DD format
  Output: absolute file path to Daily_Current_Affairs_Briefing.pdf
  Failure: FileNotFoundError if exports/ not writable; empty PDF if no
articles for date
```

### safe\_mode.py

```
health_check() → HealthStatus
  Input:  none
  Output: HealthStatus enum: HEALTHY | DEGRADED_API | DEGRADED_FEED |
DEGRADED_BOTH
  Side effect: logs DEGRADED status to experiment_runs
  Failure: never raises – returns DEGRADED_BOTH on internal error

is_safe_mode_active() → bool
  Output: True if last health_check returned any DEGRADED state
  Note:   Re-runs health_check on each call (stateless)

get_fallback_questions(subject_id: str, count: int) → List[Dict]
  Input:  subject_id, count ≤ 30
  Output: list of question dicts from static_assets/pyq/prelims/ flat JSON
files
  Failure: GenerationFailure if static pool is empty or missing
```

---

# Phase 5 Interfaces

## main.py

All routes defined in Part VIII of the master documentation. Interfaces are HTTP request/response contracts.

## benchmark\_runner.py

```
run_generation_benchmark() → BenchmarkResult
    Input:  reads from static_assets/pyq/ (held-out subset)
    Output: BenchmarkResult {alignment_score: float, sample_size: int,
drift_detected: bool}
    Side effect: writes to experiment_runs

run_evaluation_benchmark() → BenchmarkResult
    Input:  reads from static_assets/topper_copies/
    Output: BenchmarkResult {mean_score: float, below_threshold_count: int,
drift_detected: bool}
    Side effect: writes to experiment_runs
```

# Section 6: Risks & Coupling Analysis

## Coupling Violations Remaining After Corrections

| Violation                                       | Severity   | Status                                                                  |
|-------------------------------------------------|------------|-------------------------------------------------------------------------|
| generator.py calls rag_store.py directly        | LOW        | Accepted — context data too large to route through orchestrator cleanly |
| generator.py imports math_utils.py              | ACCEPTABLE | Utility import only                                                     |
| benchmark_runner.py calls evaluator.py directly | ACCEPTABLE | Offline tool, not API server                                            |

## Modules Correctly Using Orchestrator Pattern (After Corrections)

All runtime intelligence modules are called through main.py. No spoke-to-spoke calls in the API path except the two accepted exceptions above.

## Risk Register

| Risk                                        | Phase   | Probability               | Impact | Mitigation                                  |
|---------------------------------------------|---------|---------------------------|--------|---------------------------------------------|
| Memory decay formula produces bad intervals | Phase 1 | HIGH if formula not fixed | HIGH   | Apply C-03 normalization before any testing |

| Risk                                                  | Phase   | Probability                       | Impact      | Mitigation                                                                 |
|-------------------------------------------------------|---------|-----------------------------------|-------------|----------------------------------------------------------------------------|
| P_w produces negative values                          | Phase 1 | HIGH if not clamped               | MEDIUM      | Apply C-04 clamping in <code>math_utils.py</code>                          |
| ChromaDB collections empty at generation time         | Phase 3 | MEDIUM (manual provisioning step) | MEDIUM      | Critic Agent degrades gracefully; static pool as fallback                  |
| Cloud API schema failures despite Pydantic            | Phase 3 | LOW–MEDIUM                        | HIGH        | Retry up to 3 times; fallback to static pool after 3 failures              |
| Critic Agent retry storm (3 retries × 3 workers)      | Phase 3 | LOW                               | HIGH (cost) | Monitor <code>critic_retry_count</code> ; alert if >2 retries per question |
| Topper copy benchmark has no <code>question_id</code> | Phase 5 | CONFIRMED                         | MEDIUM      | C-06 resolution: optional blueprint parameter in evaluator                 |
| Static PYQ pool not provisioned before Phase 4        | Phase 4 | HIGH (manual step)                | HIGH        | Gate Phase 4 testing behind PYQ provisioning verification                  |
| APScheduler crash loses archival jobs                 | Phase 5 | LOW                               | LOW         | Accepted for V1. Data accumulates but is not corrupted.                    |
| <code>score_percentage</code> dual semantics bug      | Phase 3 | MEDIUM if not normalized          | MEDIUM      | C-01 resolution: always normalize to [0.0, 1.0] before write               |

## Section 7: Missing Critical Components

The following components are **absent from the architecture** and must be added before their dependent modules can be correctly implemented. All are required — none are nice-to-have.

### M-01 — `math_utils.py` (Required Before Phase 3)

**Missing:** All mathematical functions are currently attributed to `generator.py` and `main.py` without a centralized module.



**Why critical:** Elo calculation is called from `/submit - answer` (in `main.py`), not during generation. Memory decay is called after every attempt (in `main.py`). Neither belongs in `generator.py`. Without centralization, these functions are duplicated or misplaced.

**Resolution:** New module `math_utils.py` in Phase 1. Full interface defined in Section 5.

---

## M-02 — `expected_seconds` in Config (Required for `P_w` Formula)

**Missing:** `P_w` formula references `expected_seconds` but this value has no source.

**Why critical:** Formula cannot be implemented without this value.  $\Delta T = |\text{response\_duration} - \text{expected\_duration}|$  requires `expected_duration` per question type.

**Resolution:** Add `test_pacing` block to `calibration_config.yaml` as defined in C-02.

---

## M-03 — `consecutive_stable_attempts` in `student_profile` (Required for Baseline Elo Update)

**Missing:** `baseline_elo_rating` update rule requires tracking 20 consecutive stable attempts. No counter exists.

**Why critical:** Without this field, the baseline update rule cannot be implemented. Baseline would either never update (static for life) or update incorrectly.

**Resolution:** Add `consecutive_stable_attempts` (Integer, default 0) to `student_profile` as defined in C-11.

---

## M-04 — `source_type` in `backlog_queue` (Required for Composition Step 4)

**Missing:** Composition Step 4 applies backlog within each content category (static vs CA). The existing `topic_type` field ('GS'/'CSAT') does not encode content source.

**Why critical:** Without this field, the category-level backlog allocation in Step 4 cannot be executed correctly. All backlog items would route to the same content pool.

**Resolution:** Add `source_type` (Text: 'STATIC' or 'CA') to `backlog_queue` as defined in C-09.

---

## M-05 — `I_base` Update Formula (Required for Memory Decay Correctness)

**Missing:** `base_stability_index` in `topic_progress` is described as "updated after each review" but no update formula exists.

**Why critical:** Without an update formula, `I_base` remains permanently at 3.0 for all topics. The memory decay formula then degenerates to a fixed-coefficient calculation with no learning from history.

**Resolution:** Define in `math_utils.py`: `update_stability_index(I_base, I_next) → float`. Apply after every attempt in the `/submit-answer` handler.

---

## Section 8: Questions Before Implementation

The following questions require explicit answers before module generation begins. They are not assumptions — they are unresolved decisions.

---

### Q1 — Static PYQ File Format for Safe Mode

`safe_mode.py` reads directly from `static_assets/pyq/prelims/` JSON files without ChromaDB. The architecture states these follow `GeneratedQuestionSchema` format. But these are historical UPSC questions — they were not generated by this system and may not have been processed through the schema (they lack `trap_data` and `explanation_data`).

Options:

- (a) Require PYQ files to be manually enriched to `GeneratedQuestionSchema` format before use
- (b) Define a simpler `SafeModePYQSchema` with only `question_text`, `options`, `correct_key`, `difficulty_tier` for safe mode serving
- (c) Safe mode serves raw questions with no explanation or trap data (display-only)

Which approach?

---

### Q2 — Topper Copy Blueprint for Benchmark

`benchmark_runner.py` passes topper copies to `evaluator.py` with `blueprint=some_dict`. But where do these blueprints come from? Topper copies are plain text files with no associated blueprint.

Options:

- (a) Topper copy files include an embedded blueprint header in a structured format
- (b) Benchmark runner generates a blueprint on-the-fly via a Cloud API call before evaluation
- (c) Evaluation benchmark skips Pass 1 (blueprint check) entirely and runs Pass 2 only, noting reduced diagnostic value

Which approach?

---

### Q3 — Pacing Drift Index Response in API

When `/submit-answer` detects pacing standard deviation  $> 0.40$ , the architecture says "inject an anchored review sprint or recommend a mandatory break." In an API context, this is a response flag.

Clarify: should the `/submit-answer` response simply include a `psychological_drift_warning: bool` field for the client to act on, or should the API itself modify the next test composition (e.g., by injecting a forced easy-question session)?

This determines whether the drift response is a client responsibility or a server-side composition change.

---

### Q4 — Session ID Lifecycle

The architecture generates a `session_id` UUID when a test is created via `/generate-test/prelims`. All `/submit-answer` calls for that session include this ID. But when does a session end? Is it time-based, question-count-based, or explicitly closed?

If session IDs are never explicitly closed, `attempt_history` could accumulate orphaned `session_id` references. For the pacing std dev calculation, do we query all attempts ever made in that session (including past runs), or only attempts in the current sitting?

---

### Q5 — `document_ingestor` Trigger Mechanism

`scraper.py::document_ingestor()` is event-triggered (on RBI/Budget publication). The architecture says it "runs on event trigger." But what actually triggers it in the local-first system?

Options:

- (a) A dedicated API endpoint `POST /ingest-document` that accepts a URL as input
- (b) A file watcher that detects new files dropped into a local directory
- (c) Manual script execution only

For V1, which mechanism?

---

*End of Implementation Roadmap*

*Architecture locked. Five phases defined. All contradictions resolved or flagged.*

*Await instruction before module generation begins.*